

Week 2

DSA Unit 2 – Mathematical & Bitwise Foundations

1. Mathematical Algorithms

(a) GCD (Greatest Common Divisor)

- Largest integer that divides both numbers.
- **Euclidean Algorithm** is the most efficient.

Idea:

- $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$
- Stop when $b = 0$.

Example:

```
gcd(48, 18)
= gcd(18, 48 % 18)
= gcd(18, 12)
= gcd(12, 6)
= gcd(6, 0)
= 6
```

Pseudocode:

```
function gcd(a, b):
    while b != 0:
        temp = b
        b = a % b
        a = temp
    return a
```

Python Code:

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
```

(b) LCM (Least Common Multiple)

- Smallest positive integer divisible by both numbers.
- Formula:

$$\text{LCM}(a,b)=a \times b / \text{GCD}(a,b) \quad \text{LCM}(a, b) = \frac{a \times b}{\text{GCD}(a, b)}$$

Example:

- $\text{LCM}(12, 18) = (12 \times 18) / \text{GCD}(12, 18) = 216 / 6 = 36$

(c) Prime Numbers

- A number greater than 1 with no divisors except 1 and itself.

Naive Check: Loop from 2 to $n-1 \rightarrow O(n)$

Optimized: Loop till $\sqrt{n} \rightarrow O(\sqrt{n})$

Python Example:

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

(d) Modular Arithmetic

- Deals with remainders when dividing numbers.

- Useful in **cryptography, hashing, competitive programming**.

Properties:

- $(a + b) \% m = (a \% m + b \% m) \% m$
- $(a \times b) \% m = (a \% m \times b \% m) \% m$
- $(a - b) \% m = (a \% m - b \% m + m) \% m$

Example:

- $(7 \times 8) \% 5 = 56 \% 5 = 1$
 - $(7 \% 5 \times 8 \% 5) \% 5 = (2 \times 3) \% 5 = 6 \% 5 = 1$ 
-

2. Bitwise Operations

- Operates directly on **binary representations** of numbers.
- Very **fast and memory-efficient**.

Common Operators:

1. **AND (&)** → 1 if both bits are 1.

- $5 \& 3 = (101 \& 011) = 001 = 1$

2. **OR (|)** → 1 if at least one bit is 1.

- $5 \mid 3 = (101 \mid 011) = 111 = 7$

3. **XOR (^)** → 1 if bits are different.

- $5 \wedge 3 = (101 \wedge 011) = 110 = 6$

4. **NOT (~)** → Flips all bits (in Python: $\sim x = -(x+1)$).

5. **Shift Operators:**

- Left Shift (<<) → multiply by 2^k

$$5 << 1 = 10$$

- Right Shift (>>) → divide by 2^k

$$10 >> 1 = 5$$

Useful Bitwise Tricks:

1. Check if number is even/odd:

- `n & 1` → 0 (even), 1 (odd)

2. Swap two numbers without temp variable:

```
a, b = 5, 7
a = a ^ b
b = a ^ b
a = a ^ b
# a=7, b=5
```

3. Check if number is power of 2:

- `n > 0 and (n & (n-1)) == 0`

4. Count set bits (Hamming Weight):

- Brian Kernighan's Algo: repeatedly remove lowest set bit.

```
def count_bits(n):
    count = 0
    while n:
        n &= (n - 1)
        count += 1
    return count
```

3. Number Theory Basics

(a) Modular Properties (recap above)

(b) Fast Exponentiation (Binary Exponentiation / Power Mod)

- Efficiently computes `a^b % m` in $O(\log b)$.

Idea:

- Break power using binary form.
- Example: $3^{13} = 3^{(1101)} \rightarrow 3^8 \times 3^4 \times 3^1$

Python Code:

```
def power(a, b, m):
    res = 1
    a = a % m
    while b > 0:
        if b & 1: # if b is odd
            res = (res * a) % m
        a = (a * a) % m
        b >>= 1
    return res
```

(c) Sieve of Eratosthenes (Prime Generation)

- Finds all primes $\leq n$ in $O(n \log \log n)$.

Idea:

- Start with 2, mark all multiples as non-prime.
- Continue till $\sqrt{n}$.

Python Code:

```
def sieve(n):
    prime = [True] * (n+1)
    prime[0] = prime[1] = False
    p = 2
    while p*p <= n:
        if prime[p]:
            for i in range(p*p, n+1, p):
                prime[i] = False
            p += 1
    return [i for i, is_prime in enumerate(prime) if is_prime]
```

Example:

`sieve(30)` → `[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]`
